

苏州大学实验报告

院系	计算机学院	年级专业	21 计科	姓名	方浩楠	学号	2127405048
课程名称	大数据管理					成绩	
指导教师	王晶晶	同组实验者	无		实验日期	2024. 4. 18	

实验名称 实验 3: 借助开源 TensorFlow 工具的词向量训练

一. 实验目的

借助开源 TensorFlow 工具的词向量训练

- 对 THE TRAGEDY OF ROMEO AND JULIET (罗密欧 与朱丽叶)英文小说中词进行 word Embedding 训练, 获得每个词的向量表示。(word Embedding 获得可以调用 TensorFlow 包)
- 功能: 输入“ROMEO”单词, 输出获得 5 个与其最相似的词(降序排列)及其向量值, 例如: juliet, [0.1,0.2]

二. 实验内容

利用 TensorFlow 进行《罗密欧与朱丽叶》文本的词向量训练及相似词查找

三. 实验步骤和结果

实验环境:

```
• .venvfanghaonan@fanghaonandeMacBook-Pro experiment3 % neofetch
      'c.
      ,xNMM.
      .OMMMMo
      OMMM0,
      .;lodd0:' loolloddol;.
      cKMMMMMMMMMMNWMMMMMMMMM0:
      .KMMMMMMMMMMMMMMMMMMMMMMWd.
      XMMMMMMMMMMMMMMMMMMMMMMX.
      ;MMMMMMMMMMMMMMMMMMMMMMM:
      :MMMMMMMMMMMMMMMMMMMMMM:
      .MMMMMMMMMMMMMMMMMMMMMMX.
      kMMMMMMMMMMMMMMMMMMMMMMWd.
      .XMMMMMMMMMMMMMMMMMMMMMMk
      .XMMMMMMMMMMMMMMMMMMMMMMK.
      kMMMMMMMMMMMMMMMMMMMMMMd
      ;KMMMMMMMWXXWMMMMMMMMK.
      .COOC,. .,COO:.

      fanghaonan@fanghaonandeMacBook-Pro.local
      -----
      OS: macOS 14.4 23E214 arm64
      Host: Mac14,9
      Kernel: 23.4.0
      Uptime: 2 days, 21 hours, 22 mins
      Packages: 15 (brew)
      Shell: zsh 5.9
      Resolution: 1512x982
      DE: Aqua
      WM: Quartz Compositor
      WM Theme: Blue (Light)
      Terminal: vscode
      CPU: Apple M2 Pro
      GPU: Apple M2 Pro
      Memory: 2989MiB / 16384MiB
```

```
• .venvfanghaonan@fanghaonandeMacBook-Pro experiment3 % python -V
Python 3.9.6
```

```

Python 3.9.0
.venvfanghaonan@fanghaonandeMacBook-Pro experiment3 % pip list
Package                                Version
-----
absl-py                                2.1.0
astunparse                             1.6.3
certifi                                2024.2.2
chardet                                5.2.0
charset-normalizer                      3.3.2
flatbuffers                             24.3.25
gast                                    0.5.4
google-pasta                            0.2.0
grpcio                                  1.63.0
h5py                                    3.11.0
idna                                    3.7
importlib_metadata                     7.1.0
joblib                                  1.4.2
keras                                    3.3.3
libclang                                18.1.1
Markdown                                3.6
markdown-it-py                         3.0.0
MarkupSafe                             2.1.5
mdurl                                   0.1.2
ml-dtypes                              0.3.2
namex                                   0.0.8
numpy                                   1.26.4
opt-einsum                             3.3.0
optree                                  0.11.0
packaging                              24.0
pip                                     24.0
protobuf                               4.25.3
Pygments                               2.18.0
requests                               2.31.0
rich                                    13.7.1
scikit-learn                           1.4.2
scipy                                   1.13.0
setuptools                             68.2.0
six                                     1.16.0
tensorboard                            2.16.2
tensorboard-data-server                 0.7.2
tensorflow                              2.16.1
tensorflow-io-gcs-filesystem            0.37.0
termcolor                              2.4.0
threadpoolctl                          3.5.0
typing_extensions                      4.11.0
urllib3                                 2.2.1
Werkzeug                               3.0.3
wheel                                   0.41.2

```

requirements.txt 中的内容:

```

1 tensorflow
2 numpy
3 scikit-learn
4 chardet

```

运行项目之前,安装项目所需要的包:

```
pip install -r requirements.txt
```

项目结构图:

```
(.venv) fanghaonan@fanghaonandeMacBook-Pro experiment3 % tree .  
.  
├── data  
│   ├── convert_encoding.py  
│   ├── preprocessed_data.pkl  
│   ├── romeo_and_juliet.txt  
│   ├── word2vec_model.h5  
│   └── word_vectors.npy  
├── main.py  
├── requirements.txt  
├── scripts  
│   ├── find_similar.py  
│   ├── preprocess.py  
│   └── train_model.py  
├── 实验报告模板.docx  
└── 实验要求.md  
  
3 directories, 13 files
```

首先,由于下载的罗密欧与朱丽叶的 txt 文件的编码方式不是 UTF-8,因此编写代码来修改 txt 文件的编码格式:

convert_encoding.py:

```
import chardet  
  
# 检查文件编码  
  
with open('romeo_and_juliet.txt', 'rb') as f:  
    raw_data = f.read()  
    result = chardet.detect(raw_data)  
    encoding = result['encoding']  
    print(f"File encoding: {encoding}")  
  
# 转换为 utf-8 编码  
  
if encoding.lower() != 'utf-8':  
    with open('romeo_and_juliet.txt', 'r', encoding=encoding) as f:  
        text = f.read()
```

```
with open('romeo_and_juliet.txt', 'w', encoding='utf-8') as f:
    f.write(text)
print("File has been converted to utf-8 encoding")
```

然后进行数据预处理, 在 scripts/preprocess.py 中进行数据预处理, 读取文本文件并进行清理和标记。

preprocess.py:

```
import re
import string
from collections import Counter
from tensorflow.keras.preprocessing.text import Tokenizer
import sys
import pickle

def preprocess_text(file_path: str) -> tuple[list, dict, int]:
    with open(file_path, 'r', encoding='utf-8') as file:
        text = file.read()

    text = text.lower()
    text = re.sub(r'[%s]' % re.escape(string.punctuation), '', text)
    words = text.split()

    word_counts = Counter(words)
    words = [word for word in words if word_counts[word] > 1]

    tokenizer = Tokenizer()
    tokenizer.fit_on_texts([words])
    sequences = tokenizer.texts_to_sequences([words])[0]
    word_index = tokenizer.word_index

    vocab_size = len(word_index) + 1

    return sequences, word_index, vocab_size

if __name__ == "__main__":
    input_file = sys.argv[1]
    output_file = sys.argv[2]
    sequences, word_index, vocab_size = preprocess_text(input_file)

    with open(output_file, 'wb') as f:
        pickle.dump((sequences, word_index, vocab_size), f)
```

训练词向量模型:

在 scripts/train_model.py 中进行词向量模型的训练。

train_model.py:

```
import tensorflow as tf
import numpy as np
import pickle
import sys
from typing import List, Tuple, Dict

def generate_skipgram_data(sequences: List[int], window_size: int, vocab_size: int) ->
    Tuple[np.ndarray, np.ndarray]:
    couples: List[List[int]] = []
    labels: List[int] = []
    for i, word_id in enumerate(sequences):
        start = max(0, i - window_size)
        end = min(len(sequences), i + window_size + 1)
        for j in range(start, end):
            if i != j:
                couples.append([word_id, sequences[j]])
                labels.append(1)
    return np.array(couples), np.array(labels)

if __name__ == "__main__":
    input_file: str = sys.argv[1]
    model_file: str = sys.argv[2]
    vectors_file: str = sys.argv[3]

    with open(input_file, 'rb') as f:
        sequences, word_index, vocab_size = pickle.load(f)

    sequences: List[int]
    word_index: Dict[str, int]
    vocab_size: int

    window_size: int = 2
    couples, labels = generate_skipgram_data(sequences, window_size, vocab_size)

    print(f"Number of training samples: {len(couples)}")
    print(f"First 5 couples: {couples[:5]}")

    if couples.size == 0:
```

```

        raise ValueError("No training data generated. Check your data and preprocessing
steps.")

embedding_dim: int = 100

target_input = tf.keras.layers.Input(shape=(1,), dtype='int32')
context_input = tf.keras.layers.Input(shape=(1,), dtype='int32')

embedding = tf.keras.layers.Embedding(input_dim=vocab_size, output_dim=embedding_dim,
name='embedding')
target_embedding = embedding(target_input)
context_embedding = embedding(context_input)

dot_product = tf.keras.layers.Dot(axes=-1)([target_embedding, context_embedding])
dot_product = tf.keras.layers.Reshape((1,))(dot_product)
output = tf.keras.layers.Dense(1, activation='sigmoid')(dot_product)

model = tf.keras.models.Model(inputs=[target_input, context_input], outputs=output)
model.compile(loss='binary_crossentropy', optimizer='adam')

model.fit([couples[:, 0], couples[:, 1]], labels, epochs=5, batch_size=64)

model.save(model_file)
np.save(vectors_file, model.get_layer('embedding').get_weights()[0])

```

查找相似词

在 scripts/find_similar.py 中实现查找与特定单词最相似的词及其向量值。

ind_similar.py:

```

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import pickle
import sys

if __name__ == "__main__":
    input_file = sys.argv[1]
    vectors_file = sys.argv[2]

    with open(input_file, 'rb') as f:
        sequences, word_index, vocab_size = pickle.load(f)

    word_vectors = np.load(vectors_file)

```

```

def find_similar_words(target_word, word_index, word_vectors, top_n=5):
    target_index = word_index.get(target_word)
    if not target_index:
        return []

    target_vector = word_vectors[target_index].reshape(1, -1)
    similarities = cosine_similarity(target_vector, word_vectors)[0]

    similar_indices = similarities.argsort()[-top_n:-1][::-1]
    similar_words = [(word, similarities[index]) for word, index in word_index.items() if
index in similar_indices]

    return similar_words

similar_words = find_similar_words('romeo', word_index, word_vectors)

for word, similarity in similar_words:
    print(f"Word: {word}, Similarity: {similarity}, Vector:
{word_vectors[word_index[word]]}")

```

然后运行 main.py:

main.py:

```

import os

if __name__ == "__main__":
    project_dir = os.path.dirname(os.path.abspath(__file__))
    data_dir = os.path.join(project_dir, 'data')
    scripts_dir = os.path.join(project_dir, 'scripts')

    # 预处理

    os.system(f'python {scripts_dir}/preprocess.py {data_dir}/romeo_and_juliet.txt
{data_dir}/preprocessed_data.pkl')

    # 训练模型

    os.system(f'python {scripts_dir}/train_model.py {data_dir}/preprocessed_data.pkl
{data_dir}/word2vec_model.h5 {data_dir}/word_vectors.npy')

    # 查找相似词

```

```
os.system(f'python {scripts_dir}/find_similar.py {data_dir}/preprocessed_data.pkl  
{data_dir}/word_vectors.npy')
```

运行结果:

```
/Users/fanghaonan/File/study/Soochow_University/Big_Data_Management/experiment3/.venv/bin/python /Users/fanghaonan/File/study/Soochow_University/Big_Data_Management/experiment3/  
/Users/fanghaonan/File/study/Soochow_University/Big_Data_Management/experiment3/.venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supp  
warnings.warn(  
/Users/fanghaonan/File/study/Soochow_University/Big_Data_Management/experiment3/.venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supp  
warnings.warn(  
Number of training samples: 94730  
First 5 couples: [[ 2  6]  
[ 2 31]  
[ 6  2]  
[ 6 31]  
[ 6 1]]  
Epoch 1/5  
1481/1481 ----- 2s 849us/step - loss: 0.3266  
Epoch 2/5  
1481/1481 ----- 1s 833us/step - loss: 0.0031  
Epoch 3/5  
1481/1481 ----- 1s 840us/step - loss: 5.3066e-04  
Epoch 4/5  
1481/1481 ----- 1s 842us/step - loss: 1.5482e-04  
Epoch 5/5  
1481/1481 ----- 1s 849us/step - loss: 5.3680e-05
```

```
Word: that, Similarity: 0.9837651252746582, Vector: [-0.29130858 -0.34876123  0.34900966  0.08172053 -0.3168362 -0.35225943  
-0.03670983  0.1369016 -0.2888717 -0.03876976  0.2374117  0.3486315  
-0.29988348  0.42711982  0.15602274  0.040821  0.34581956  0.3718582  
0.31822492  0.23685329  0.40655085  0.14291565 -0.29775417  0.4389211  
0.11434452  0.32525215  0.28241563 -0.14295606  0.27865526  0.05666506  
0.13047472  0.41103214 -0.32607886  0.27931997 -0.40507337 -0.3199251  
-0.3795577 -0.21780759 -0.30346188  0.19252302 -0.44465914 -0.29856938  
0.35921174  0.20990478  0.32236505  0.03245266  0.4250796  0.10109747  
0.20808355  0.35437453  0.36937612 -0.37526667 -0.21577321 -0.33626238  
0.39308172 -0.3814367  0.43382958 -0.2778584  0.4237629 -0.05613505  
0.3264932  0.4421897  0.3478578 -0.38046402 -0.2564605 -0.36611626  
0.39549205 -0.20676267  0.46383005 -0.3473386  0.17379273  0.38783005  
-0.00215913  0.44974947 -0.3980473  0.42034474 -0.32785285  0.1600185  
-0.32757452 -0.35536173 -0.03869929  0.28295386 -0.2603852  0.43569133  
0.29811043  0.28761926 -0.22660533  0.43783775  0.43082485 -0.35471693  
-0.20619012  0.44761074  0.33382767  0.29805487 -0.31303322 -0.34550467  
0.3437952  0.1777842  0.42426708  0.37820777]  
Word: not, Similarity: 0.9844006299972534, Vector: [-0.32137042 -0.4193752  0.36534297  0.00419443 -0.3094066 -0.28651464  
-0.06169606  0.1991571 -0.26378292  0.00214024  0.20433956  0.33256763  
-0.27398065  0.28441328  0.26541215  0.02887993  0.23152912  0.35273883  
0.2790843  0.3214252  0.36537066  0.19316861 -0.33341232  0.33639503  
0.15709297  0.31598106  0.31524885 -0.04543453  0.3370121  0.00368002  
0.05435178  0.40512022 -0.35804832  0.25694323 -0.3549098 -0.35242593  
-0.35855514 -0.18153003 -0.2306834  0.14212123 -0.36114094 -0.20779584  
0.37981305  0.12651241  0.28995627  0.03992514  0.42504695  0.01282942  
0.1821394  0.3957284  0.2958513 -0.41248405 -0.19942044 -0.31147066  
0.29225734 -0.35811925  0.36500818 -0.29497707  0.3149743  0.01493283  
0.35336363  0.421621  0.34603333 -0.3315294 -0.27925235 -0.3288771  
0.3147909 -0.21732673  0.37779802 -0.38452315  0.01858087  0.3348244  
-0.06182163  0.37503245 -0.34039542  0.30310768 -0.354192  0.19576047  
-0.28419268 -0.28031157 -0.02617539  0.25837702 -0.33516422  0.38087597  
0.34500825  0.26578468 -0.06923138  0.39691088  0.37724435 -0.35939425  
-0.2919169  0.35736358  0.37066913  0.23611306 -0.3061877 -0.36564904  
0.39630976  0.14459193  0.4212734  0.3922291 ]
```

```
Word: be, Similarity: 0.9819134473800659, Vector: [-0.34796745 -0.33132493 0.37922123 0.06000757 -0.28718358 -0.286001
-0.05085429 0.19340685 -0.24832736 0.01230814 0.1850499 0.3635729
-0.24605028 0.35186085 0.24115004 0.06829581 0.2529264 0.3675467
0.30426806 0.32586855 0.3972728 0.13457245 -0.3138326 0.42011917
0.13396162 0.22676082 0.30541912 0.01521125 0.33518454 0.04690528
0.07926305 0.30493927 -0.21881631 0.28257236 -0.40822285 -0.40687406
-0.4050238 -0.24524064 -0.30910337 0.17387792 -0.3987904 -0.3139879
0.33977595 0.20785342 0.34712362 -0.03588112 0.4004527 0.11061578
0.1881272 0.31655607 0.3581763 -0.36506796 -0.27891335 -0.35786088
0.3375902 -0.3642623 0.3534536 -0.16815023 0.34600118 0.03784278
0.36098582 0.39998952 0.37659687 -0.32823512 -0.23544027 -0.2851457
0.29362935 -0.14798833 0.38357294 -0.3364725 0.03595274 0.34350035
-0.06423607 0.38921818 -0.3013083 0.2725925 -0.36201477 0.21985132
-0.34148195 -0.32528344 0.01974369 0.23051171 -0.33192372 0.40197197
0.2792829 0.2769161 -0.16473155 0.40452003 0.3957096 -0.33102104
-0.22227217 0.3495126 0.30015913 0.3227557 -0.26108074 -0.33323514
0.30165097 0.14719778 0.42802355 0.35083464]
Word: what, Similarity: 0.9806060791015625, Vector: [-0.2556378 -0.36840373 0.33612305 0.04085904 -0.3156229 -0.29779327
-0.04358812 0.2104545 -0.31914145 -0.08896068 0.26524827 0.31763712
-0.24953803 0.29825774 0.23076387 0.10712463 0.2558938 0.21876514
0.31130773 0.2520043 0.36987576 0.19155414 -0.31835222 0.27461523
0.12157393 0.21233995 0.19994515 -0.173578 0.3226563 0.01542031
0.12331529 0.38557294 -0.33907574 0.27847537 -0.3932696 -0.35436556
-0.31580362 -0.23545247 -0.20523465 0.19855222 -0.32976025 -0.3109391
0.34386063 0.15299109 0.2535765 0.15991178 0.31632438 0.01379859
0.24424018 0.3727296 0.3822409 -0.30664632 -0.18851209 -0.30413097
0.3447139 -0.30716994 0.38353035 -0.21323457 0.30050698 0.09822827
0.32378155 0.34842014 0.23894581 -0.3406239 -0.29871643 -0.31398568
0.34399298 -0.175378 0.32953125 -0.31152517 0.13097146 0.3811404
-0.1127018 0.33684325 -0.2858275 0.3279532 -0.39184082 0.26914722
-0.30727288 -0.2582751 -0.1002556 0.28482047 -0.27772164 0.36627078
0.27825713 0.30411014 -0.10487388 0.3891484 -0.29901624 -0.33587953
-0.3386771 0.24917726 0.27834496 0.26910603 -0.20600604 -0.3683187
0.36247194 0.12112771 0.35553804 0.33337528]
0.36247194 0.12112771 0.35553804 0.33337528]
Word: rom, Similarity: 0.9853156805038452, Vector: [-0.19684008 -0.37795573 0.33543468 -0.04606728 -0.33685553 -0.31719112
-0.1224179 0.14278944 -0.28277615 0.012644 0.18960866 0.36800802
-0.3054168 0.3513568 0.2513593 0.11090577 0.2964616 0.25771526
0.3747779 0.25999984 0.36470574 0.19852965 -0.30606753 0.3379245
0.02927892 0.28073743 0.24770533 -0.12758501 0.26858503 -0.04640966
0.12499394 0.35229266 -0.31124017 0.28173906 -0.4024976 -0.33483863
-0.33288264 -0.24855769 -0.21533051 0.09934782 -0.38323548 -0.27780938
0.3559456 0.14119627 0.34465238 0.04029762 0.38872463 -0.05408372
0.19478568 0.40637273 0.37355354 -0.34741333 -0.16655749 -0.29342213
0.26497048 -0.3785499 0.3791303 -0.2443689 0.26662928 0.11762968
0.31955034 0.360236 0.3302492 -0.29901668 -0.26334307 -0.33920103
0.3498232 -0.21544667 0.34666196 -0.34935775 0.03542437 0.35496846
-0.10174491 0.31929892 -0.2870951 0.34371778 -0.39147374 0.1812293
-0.3148608 -0.25305519 -0.00807148 0.259555 -0.30692378 0.39153606
0.27264494 0.30257404 -0.12207859 0.34108472 0.32498342 -0.29637876
-0.2865869 0.30169964 0.3489038 0.24220459 -0.22190557 -0.34081608
0.29216167 0.1557168 0.3802491 0.30304775]
进程已结束，退出代码为 0
```

四. 实验总结

通过本次实验，我学习了如何利用 TensorFlow 进行文本的词向量训练，并实现了查找与特定单词最相似的词及其向量值的功能。实验结果显示，模型能够成功地找到与输入单词最相似的词，并输出其向量值。这为进一步的自然语言处理和文本分析提供了基础。